

Alto Audit Report

Version 1.0

Conducted by: Kiki

September 2026

Table of Contents

1 Introduction **3**
- 1.1 About Kiki . . . 3
- 1.2 Disclaimer . . . 3
- 1.3 Risk classification . . . 3
  - 1.3.1 Impact . . . 3
  - 1.3.2 Likelihood . . . 3
  - 1.3.3 Actions required by severity level . . . 3

2 Executive Summary **4**
- 2.1 Overview . . . 4
- 2.2 Scope . . . 4
- 2.3 Issues Found . . . 4
- 2.4 Findings & Resolutions . . . 4

3 Findings **6**
- 3.1 Medium Risk . . . 6
  - 3.1.1 Full fee claim before mint repay/liquidate can brick exits . . . 6
- 3.2 Low Risk . . . 6
  - 3.2.1 Stale seedAssets results in ever increasing depositCapUsage . . . 6
  - 3.2.2 Exiting mid reward period results in extra yield for address(1) . . . 7
  - 3.2.3 Rate recovery can be reset by an inactive pool . . . 8
  - 3.2.4 Missing on-chain enforcement that the first deposit is the configured seed . . . 9

1 Introduction

1.1 About Kiki

Kiki is an Independent Security Researcher who has conducted over 80 audits with top security firms such as Enigma Dark, Cyfrin, Bail Security, Burra Sec, Guardian Audits, as well as through private engagements. View their previous work here, or reach out via Twitter or Telegram.

1.2 Disclaimer

Security Reviews are a time, resource, and expertise bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can show the presence of vulnerabilities **but not their absence**.

1.3 Risk classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

1.3.1 Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost or a functionality of the protocol is affected.
- **Low** - any kind of unexpected behaviour that's not so critical.

1.3.2 Likelihood

- **High** - direct attack vector; the cost is relatively low to the amount of funds that can be lost.
- **Medium** - only conditionally incentivized attack vector, but still relatively likely.
- **Low** - too many or too unlikely assumptions; provides little or no incentive.

1.3.3 Actions required by severity level

- **Critical** - client **must** fix the issue.
- **High** - client **must** fix the issue.
- **Medium** - client **should** fix the issue.
- **Low** - client **could** fix the issue.

2 Executive Summary

2.1 Overview

Project Name	Alto
Codebase	v1
Operating platform	Ethereum
Language	Solidity
Initial commit	[1] 2227312e23b59fd43237fe7d0f5c3570acb357e3
Remediation commit	9472f4e5ca26954ef0223c1a23049e4fe5a5a9ed
Audit methodology	Manual Review

2.2 Scope

Files and folders in scope

- contracts/lending/irm/PegResponsiveExponentialIrm.sol
- contracts/oracles/DusdPegCurveOracle/DusdPegCurveOracle.sol
- contracts/sdusd/ProtocolFeeCollector.sol
- contracts/sdusd/SDUSD.sol
- contracts/lending/AltoMintMarket.sol

2.3 Issues Found

Severity	Total Found	Resolved	Partially Resolved	Acknowledged
Critical risk	0	0	0	0
High risk	0	0	0	0
Medium risk	1	0	0	1
Low risk	4	2	0	2

2.4 Findings & Resolutions

ID	Title	Severity	Status
M-01	Full fee claim before mint repay/liquidate can brick exits	Medium	Acknowledged
L-01	Stale seedAssets results in ever increasing depositCapUsage	Low	Acknowledged
L-02	Exiting mid reward period results in extra yield for address(1)	Low	Resolved
L-03	Rate recovery can be reset by an inactive pool	Low	Acknowledged
L-04	Missing on-chain enforcement that the first deposit is the configured seed	Low	Resolved

3 Findings

3.1 Medium Risk

3.1.1 Full fee claim before mint repay/liquidate can brick exits

Severity: *Medium risk (Acknowledged)*

Context: ***AltoMintMarket.sol***:*248-264*

Description: `AltoMintMarket._repay` and `_liquidate` always `_accrueInterest()`, then if `claimableFeesAssets > 0` call `_claimInterest(0)`, which `DUSD.mint` the entire fee balance to `feeRecipient` before any repay/liquidation burn. `DUSD.mint` reverts `DUSDInvalidInput` when `minted + amount > minterConfig.ceiling`. Accrual grows `totalBorrowed` and `claimableFeesAssets` 1:1 with no ceiling check. Deploy sets `minterCeiling = debtCeiling * 1.05 / maxLiquidationLtv` (~1.3x debt ceiling). Under high IRM rates near full utilization, unclaimed (or cumulatively claimed) fees can exceed remaining headroom, so repay and liquidate revert with no burn and fees uncleared. Failed liquidations leave underwater debt uncleared while interest can keep accruing, so shortfalls can grow and threaten DUSD backing (protocol bad debt / undercollateralization) until an admin raises the minter ceiling.

Recommendation:

Do not require a successful full fee mint before exit burns. Prefer either:

1. Cap claims to headroom: mint `min(claimableFees, ceiling - minted)` (no revert on ceiling; leave residual fees for later), including in `claimInterest(0)` / PFC harvest; or
2. Remove fee claims from `_repay` / `_liquidate` and claim only via `claimInterest` / harvest.

Resolution: Acknowledged by the team.

We accept the operational requirement to maintain sufficient minting headroom for fees and interest growth.

3.2 Low Risk

3.2.1 Stale seedAssets results in ever increasing depositCapUsage

Severity: *Low risk (Acknowledged)*

Context: ***SDUSD.sol***:*130*

Description:

`depositCapUsage` subtracts only the fixed original `_seedAssets` from the exact seed deposit. NatSpec already states that released rewards, and rewards earned by the seed, consume capacity, and that usage is `totalAssets()` - that fixed seed principal. The mismatch is with the cap's purpose: limit **live** deposits. Yield behind `address(1)` is irrecoverable, yet it still raises `totalAssets` and usage. As seed

yield accumulates (or after a large unlock left on the seed), usage can hit `depositCap` while withdrawable deposits are far below it. Then `maxDeposit = 0`, new stakers cannot enter, and if free float is empty, later `notify` / atomic `distribute` can stay blocked until an admin raises the cap.

Example Scenario:

- `_seedAssets = 1e18`, seed shares at `address(1)`: `1e18`, `depositCap = 100e18`
- Depositor stakes `1e18` -> `totalSupply = 2e18`, `totalAssets = 2e18`, `depositCapUsage = 2e18 - 1e18 = 1e18` (under cap)
- Reward `R = 250e18` notifies and fully unlocks while both still hold shares -> `totalAssets = 252e18`
  - Depositor (`1e18` shares): `convertToAssets ~= 126e18` (principal + half of `R`)
  - `address(1)` (`1e18` shares): `convertToAssets ~= 126e18` (principal + half of `R`)
  - `depositCapUsage = 252e18 - 1e18 = 251e18 > 100e18`
- Depositor redeems all shares and takes ~= `126e18` Left: `totalSupply = 1e18`, `totalAssets ~= 126e18` (all behind `address(1)`) Live free-float deposits: `depositCapUsage = 126e18 - 1e18 = 125e18 > 100e18` -> `maxDeposit = 0`
- The depositor received their share of the reward; the unreachable seed yield still counted against the cap (only `_seedAssets = 1e18` was exempt). After they leave, the cap stays exceeded and new deposits / later distribution can remain stuck without an admin cap raise.

Even without an extreme `R`, usage drifts as `address(1)` accrues yield, and that gap grows over time.

Recommendation:

Include accrued yield as exempt from the live-deposit cap, e.g.:

```
 function depositCapUsage() public view returns (uint256) {
     uint256 assets = totalAssets();
-    uint256 seedAssets = _seedAssets;
+    uint256 seedAssets = convertToAssets(seedShares); // or balanceOf(address(1))
    if all locked shares should be exempt
     return assets > seedAssets ? assets - seedAssets : 0;
 }
```

Prefer `convertToAssets(seedShares)` if only the configured seed size should be ignored; `balanceOf(address(1))` also exempts later transfers into the dead lock.

Resolution: Acknowledged by the team.

We acknowledge that irrecoverable seed earnings can consume capacity and require a cap increase, including after all outside holders exit.

3.2.2 Exiting mid reward period results in extra yield for address(1)

Severity: *Low risk (Resolved)*

Context: ***SDUSD.sol***:148-151

Description:

`SDUSD.notifyRewardAmount` only requires free float at period start (`totalSupply()> balanceOf(address(1))`). A dust deposit can satisfy that check, `ProtocolFeeCollector.distribute` can book a large reward tranche `R`, and the dust holder can redeem during the lock while `totalAssets()` still excludes `lockedRewardsRemaining()`. Free float is empty again, so the unlocking `R` accrues to the irrecoverable seed at `address(1)`.

Example Scenario: - Free float empty except seed at `address(1)` - Attacker deposits dust sDUSD -> `distribute` notifies `R = 400e18` -> redeems dust while rewards are locked - Redeem returns ~dust principal (`totalAssets` unchanged at notify) - After `rewardDuration`, `totalAssets` includes `R` backing only `address(1)` -> protocol fees intended for live stakers are permanently bound to the dead lock

Recommendation:

Document clearly that exiting mid-reward period increases the value permanently received by `address(1)`.

Resolution: Resolved. The recommended fix was implemented in **9472f4e**.

3.2.3 Rate recovery can be reset by an inactive pool

Severity: *Low risk (Acknowledged)*

Context: ***PegResponsiveExponentialIrm.sol**:275*

Description:

`PegResponsiveExponentialIrm` uses Curve `price_oracle` (EMA) for stress enter/exit, but rejects a sample when `block.timestamp - ma_last_time > maxOracleAge`. Trades update both the EMA state and `ma_last_time`. Between trades, `ma_last_time` is frozen while `price_oracle()` may still slowly move via time projection toward the last trade's `last_price`. Alto checkpoints only read that pair; they do not refresh `ma_last_time`.

On age-stale (`oracleValid = false`), `_nextStressState` ignores price, keeps current `stressMode`, and sets `recoveryStartedAt = 0`; `_refreshBorrowRate` holds the stored rate (with stress/base floor). So after a real re-peg has started recovery, a quiet pool + later `accrueInterest` can wipe recovery and leave borrowers on >= `stressRate` even though the last trade's EMA still looks healthy. The same age gate can also skip stress **entry** if a depeg's first IRM sample arrives only after `maxOracleAge` (EMA still reflects the depeg trade, timestamp is already old) and the pre-depeg rate is held.

Example Scenario:

- Tentative config: `maxOracleAge = 30m`, `recoveryDelay = 6h`, `stressPrice = 1e18`, `recoveryPrice = 1.001e18`
- **Stuck high (primary):** Curve depeg trades + fresh `accrueInterest` -> stress on; Curve re-peg trades + fresh accrue -> `recoveryStartedAt` set; no Curve trades for >30m (`ma_last_time` stuck; EMA still ~healthy); anyone `mintMarket.accrueInterest()` -> stale -> stress kept, recovery cleared -> rate stays >= `stressRate`
- **Missed entry (secondary):** Curve depeg trades at t1; no Alto accrue until t1+31m; first accrue sees depegged EMA but stale `ma_last_time` -> no stress entry, keep `baseRate`

Recommendation:

On age-stale samples, hold the stored rate but do not clear recovery.

Resolution: Acknowledged by the team.

We acknowledge these limitations and retain the specified behavior.

3.2.4 Missing on-chain enforcement that the first deposit is the configured seed

Severity: *Low risk (Resolved)*

Context: ***SDUSD.sol***:293

Description: `SDUSD.initialize` stores a non-zero `seedShares` threshold but does not mint shares or pull DUSD. The permanent seed exemption is recorded only in `_deposit`, and only when `totalSupply()== 0`, `receiver == address(1)`, and `assets == shares == seedShares`. Nothing requires that call to be the first deposit. Any other first deposit leaves `_seedAssets == 0`; a later `deposit(seedShares, address(1))` cannot claim the exemption because supply is already non-zero. NatSpec still obligates deployment to seed `address(1)` after init; however, any gap between deployment and this initial deposit can cause seeding to fail. `notifyRewardAmount` still keys off that configured threshold, `lockedShares = balanceOf(address(1))` and requires `lockedShares >= seedShares` so a non-zero `seedShares` is enforced as the required lock size even when there are no actual locked shares yet. Integrators reading `seedShares()` still see a non-zero seed as if the permanent lock were already in place.

Recommendation:

When `totalSupply()== 0`, require `receiver == address(1)` and `assets == seedShares` (with minted `shares == seedShares`); otherwise revert.

Resolution: Resolved. The recommended fix was implemented in **9472f4e**.